CL3 Viva Theory & Code Walkthrough — Detailed Edition

CL3 Viva Theory & Code Walkthrough — Detailed Edition

A complete, *examples-first* guide to all 10 practicals of **417534: Computer Laboratory III**. For each practical you get:

1. **Aim** restated.
2. **Real-world analogy** — so the abstract idea sticks.
3. **Theory** — every concept and term explained in plain language.
4. **Worked example** — actual numbers and traces showing how the algorithm/protocol behaves step-by-step.
5. **Why this language / framework?** — explicit justification.
6. **Code walkthrough** — what each part of the code does and why.
7. **Real-world places this technology is used today.**
8. **Common viva questions, with detailed answers.**

A short *cross-cutting* Q&A appears at the very end with questions that span multiple practicals, plus a glossary.

---

0. Foundations You'll Be Asked First

Before drilling into a specific practical, examiners almost always ask broader questions. Be ready for these.

0.1 What is a "distributed system"?

A collection of independent computers connected by a network that **appears to the user as one coherent system**. They cooperate by passing messages.

Real-world analogy. A pizza delivery chain. The phone-line takes your order, the kitchen prepares the pizza, the rider delivers it. Three separate "machines" but to you it feels like one company. None of them shares memory — they all coordinate by passing messages (the printed order ticket, the rider's GPS, the receipt).

Defining properties: - **No shared memory** — every node has its own RAM. Coordination is by message passing only. - **No global clock** — clocks drift; you cannot rely on absolute timestamps. - **Partial failure** — *some* nodes can fail while others keep working. (In a single-machine program, either it crashes entirely or runs.)

Goals (and why each matters): | Goal | Meaning | Example | |——|———|———| | Transparency | Users can't tell that work is distributed | A Google search uses thousands of servers; you just type and read results | | Scalability | Add machines to handle more load | Black Friday on Amazon — auto-scale to 10× normal traffic | | Fault tolerance | One node failure doesn't bring the system down | Netflix keeps streaming when an AWS rack fails | |

Resource sharing | Many users share expensive resources | A university supercomputer used by all departments | | Concurrency | Multiple things happen at once | Twitter handles millions of tweets per minute |

0.2 Concurrency vs Parallelism vs Distribution

Term	Where?	Example
Concurrent	Multiple tasks make progress in overlapping time on possibly **one CPU** (interleaved)	A waiter taking orders from many tables — one at a time but quickly
Parallel	Multiple tasks **literally** at the same instant on **multiple cores**	Two waiters working two halves of the restaurant simultaneously
Distributed	Tasks run on **different machines** connected by a network	One chef in Pune, another in Mumbai, both serving customers in Pune via delivery

A distributed system is usually concurrent and often parallel; the converse is not necessarily true.

0.3 How processes talk to each other (IPC)

Mechanism	Where used	Example
Sockets (raw TCP/UDP)	Lowest level — everything builds on these	Web browser → web server
RPC	Function-style calls across machines	Practical 1
RMI	Method-style calls on remote objects	Practicals 2 & 9
REST / HTTP	Web APIs	Twitter API, GitHub API
gRPC	Modern high-performance RPC	Google's internal services, Netflix
Message queues	Async, decoupled communication	RabbitMQ in food-delivery apps
Shared memory	Same machine only	OS-level optimization
Pipes	Same machine only	`ls | grep .txt` in your shell

0.4 Synchronous vs Asynchronous

- **Synchronous** — caller **waits** for the answer. Easy to reason about; bad for performance if the call is slow. *Example:* you call a friend and wait for them to pick up before you can talk.

- **Asynchronous** — caller fires the request and continues; the result arrives later. *Example:* you text the friend and continue with your day; the reply arrives whenever.

RMI and RPC are usually synchronous. WebSockets, message queues, and JavaScript `async/await` are async.

0.5 Stateful vs Stateless

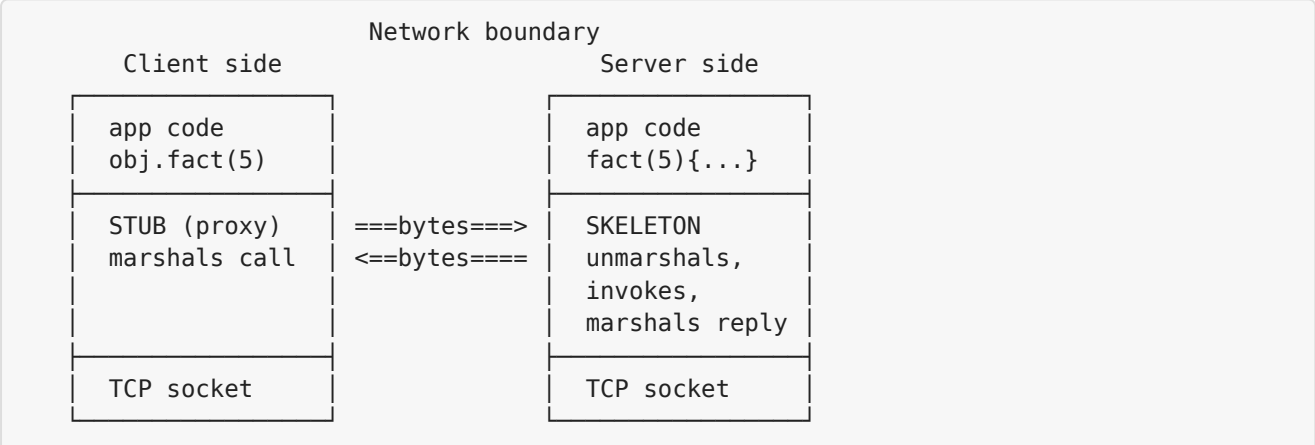
- **Stateful** — the server remembers things across calls. *Example:* the Hotel Booking server (Practical 9) keeps the bookings map.
- **Stateless** — every request carries everything it needs; the server forgets you between calls. *Example:* a Google search; each query is independent. Stateless services are far easier to scale and load-balance.

0.6 Marshalling / Serialization

Converting an in-memory object into a sequence of bytes that can travel over a wire. Reverse: **unmarshalling**.

Concrete picture. Suppose you call `obj.fact(5)` over the network. The integer 5 (4 bytes in memory) plus the method name `"fact"` plus a request id all need to be written to a TCP socket. The server receives those bytes, **unmarshals** them back into a method call, executes it, and **marshals** the return value (`120`) back as bytes.

0.7 Stub & Skeleton — the magicians behind RPC/RMI



- **Stub** = client-side dummy that *looks like* the remote object. Same methods, but each method body internally serializes args, sends them over TCP, blocks for reply, deserializes the reply.
- **Skeleton** = server-side counterpart. Listens on a socket, decodes incoming calls, invokes the real method, encodes the result.

In modern Java RMI (since Java 1.5), the skeleton is generated automatically; you don't write or compile it.

0.8 Failure semantics — what happens when the network breaks?

Semantic	Meaning	Used by
At-most-once	Call executes 0 or 1 times	**Java RMI**, modern RPCs
At-least-once	Call may execute multiple times (retries)	Idempotent REST endpoints
Exactly-once	Call executes exactly 1 time	Theoretical ideal; approximated with idempotency keys
Maybe	No guarantee	Fire-and-forget UDP

Why this matters for your hotel server. `bookRoom(5, "Alice")` is **not idempotent** — calling it twice should NOT book two rooms. So we need at-most-once semantics. Java RMI gives that.

Practical 1 — RPC: Distributed Factorial

1.1 Aim

A distributed application using **Remote Procedure Call** in Python: client sends an integer `n`, server returns `n!`.

1.2 Real-world analogy

You're at a small restaurant. You don't have a kitchen at your table — you tell the waiter "I'd like a pasta", the waiter walks the request to the kitchen, the chef cooks it, the waiter brings it back. From your perspective you "ordered pasta"; you don't know about kitchens, ovens, ingredients, or who is on shift.

That's RPC: - **You** = the client - **Kitchen** = the server - **Waiter** = the network - **Order ticket** = serialized request - **Plate of pasta** = serialized response

1.3 Theory in plain language

1.3.1 What is RPC?

Remote Procedure Call lets your program call a function that runs *on another machine* as if it were local. The framework hides: - Network sockets - Marshalling arguments - Sending bytes - Receiving bytes - Unmarshalling the result - Handling errors

You see: `result = obj.fact(5)`. Underneath, packets fly across a network.

1.3.2 The 6 steps of any RPC call

```
1. Client calls stub:        result = obj.fact(5)
2. Stub marshals:            packs ("fact", 5) into bytes
3. Bytes travel over TCP:    => 0x73 0x65 0x6e 0x74 ...
4. Server unmarshals:        decodes back to ("fact", 5)
5. Server runs the function: f = 1; for i in 1..5: f *= i  -> 120
6. Result travels back:      120 as bytes => unmarshalled => returned
```

1.3.3 RPC frameworks through history

Year	Framework	Notes
1980s	Sun RPC / ONC RPC	Used by NFS (still around)
1990s	CORBA, DCOM	OO-RPC, complex, dead now
1998	XML-RPC	XML over HTTP, evolved into SOAP
2000s	SOAP	Verbose, slow, enterprise-y
2008	Apache Thrift	Used by Facebook
2010s	**gRPC** (Google)	Protobuf, HTTP/2, used by everyone
Throughout	Pyro / Pyro4	Pythonic RPC — what we use

1.3.4 Pyro4 internals

- **Pyro** = "Python Remote Objects".
- Each remote object lives inside a **Daemon** that listens on a TCP port.
- Registration returns a **URI** like `PYRO:obj_8a2f3@localhost:39845`. That string is the only thing the client needs.
- Default serializer is **serpent** (safe; refuses arbitrary code). The older `pickle` could execute arbitrary code if used carelessly.

1.3.5 Factorial recap

`n! = 1 × 2 × 3 × … × n`, with `0! = 1`. Defined for non-negative integers. - 5! = 120 - 10! = 3,628,800 - 20! ≈ 2.4 × 10^{18} (overflows a 32-bit int) - 100! is a 158-digit number (Python handles it; Java needs `BigInteger`)

1.4 Worked example — what really happens when the client calls `fact(5)`

Imagine the server URI is `PYRO:obj_a@127.0.0.1:9000`.

Client:

```
proxy = Pyro4.Proxy("PYRO:obj_a@127.0.0.1:9000")
print(proxy.fact(5))   # prints 120
```

Step-by-step trace:

1. `Pyro4.Proxy(uri)` — no network call yet, just creates a stub object.
2. `proxy.fact(5)` — Python's special method `__getattr__` on the proxy intercepts the call.
3. Stub marshals into a serpent message:

```
{ "object": "obj_a",
  "method": "fact",
  "args":   [5],
  "kwargs": {} }
```

4. TCP connection opened to `127.0.0.1:9000`. Bytes sent.
5. Server's `Pyro4.Daemon` receives bytes, decodes, looks up object `obj_a`, finds method `fact`, calls `Factorial.fact(5)`.
6. Server runs the loop:

```
f=1
i=1: f=1
i=2: f=2
i=3: f=6
i=4: f=24
i=5: f=120
```

7. Server marshals `120`, sends back over the TCP socket.
8. Client unmarshals, returns `120` from `proxy.fact`.
9. `print(120)` displays `120`.

Total network round-trips: 1. Total serialization: 2 (one each way).

1.5 Why Python (Pyro4)?

- **Concise** — entire client+server in ~25 lines combined. Easy to demo.
- **Pure Python** — `pip install Pyro4` and you're done. Works on any OS.
- **Arbitrary-precision integers** — Python's `int` grows automatically; `100!` works without a single tweak. Java would require `BigInteger` and changes to the interface.
- **Could you do it in Java?** Yes (Sun RPC, gRPC, XML-RPC), but Pyro4 is by far the simplest way to demonstrate the *concept* of RPC end-to-end.

1.6 Code walkthrough

`server.py` — line by line

```
import Pyro4
```

Brings in the Pyro4 library.

```
@Pyro4.expose
class Factorial:
```

The `@Pyro4.expose` decorator marks the class as remotely callable. Pyro4 refuses to expose anything not explicitly decorated — a security default to prevent accidental exposure.

```
def __init__(self):
    self.calls = 0
```

A per-instance counter (so the live demo can show “I’ve been called 7 times”).

```
def fact(self, n):
    self.calls += 1
    if n < 0:
        raise ValueError("factorial undefined for negative numbers")
```

Input validation. The `ValueError` is **automatically serialized and re-raised on the client**. RPC frameworks make exceptions feel local.

```
f = 1
for i in range(1, n + 1):
    f *= i
return f
```

Iterative factorial.

```
daemon = Pyro4.Daemon()
uri = daemon.register(Factorial())
```

- `Pyro4.Daemon()` opens a TCP socket on a random free port.
- `register()` registers a `Factorial` instance with the daemon and returns a URI.

```
daemon.requestLoop()
```

Blocks forever, dispatching incoming RPC requests.

`client.py` — line by line

```
proxy = Pyro4.Proxy(uri)
proxy._pyroBind()
```

- `Pyro4.Proxy(uri)` builds a stub.
- `_pyroBind()` opens the TCP connection eagerly so we know upfront if the URI is wrong.

```
result = proxy.fact(n)
```

Looks like a local method call but is actually a network round-trip.

```
ms = (time.perf_counter() - t0) * 1000
```

Measures end-to-end latency. On localhost this is sub-millisecond; over a real network it could be 10–100 ms.

1.7 Where RPC is used in real life today

- **Google's internal services** — every microservice talks to others via gRPC.
- **Netflix** — backbone built on gRPC.
- **Kubernetes API** — grpc/protobuf for kubelet ↔ control plane.
- **Slack desktop app** — RPC between renderer and main process.
- **GitHub Actions runners** — RPC to GitHub's API.
- **Apache Hadoop** — uses internal RPC for NameNode ↔ DataNode chatter.

1.8 Likely viva questions (with detailed answers)

Q: Define RPC. A: A protocol/paradigm where a program invokes a procedure on a remote machine as if it were local. The framework hides networking, marshalling, and error handling.

Q: Difference between a normal function call and an RPC call? A: | Aspect | Local call | RPC call | |——–|———–|———-| | Time cost | nanoseconds | milliseconds (network) | | Failure mode | Bug only | Bug, network, server crash, timeout | | Arguments | Anything | Must be serializable | | State sharing | Same memory | None — pass by value usually | | Debugger | Easy | Hard — two processes |

Q: What does `@Pyro4.expose` do? A: Marks a class or method as remotely callable. Without it Pyro4 refuses access — a deliberate "deny by default" for security.

Q: What is a Pyro4 URI? A: A string `PYRO:<id>@<host>:<port>` uniquely addressing a remote object on the network. The client uses it to construct a Proxy.

Q: Synchronous or asynchronous? A: Pyro4 calls are synchronous by default. Pyro4 also supports `oneway` calls (fire-and-forget) and futures for async.

Q: How are exceptions handled across the network? A: Server serializes the exception, transmits it, client raises it locally. Pyro4 even preserves the remote stack trace.

Q: What is marshalling? A: Converting in-memory objects into a sequence of bytes that can travel over a network. The reverse is unmarshalling.

Q: Without a name server, how does the client find the server? A: It must be told the URI manually (here we copy-paste it). With a Pyro4 name server, the client could do `Pyro4.locateNS().lookup("factorial.service")`.

Q: What if the network drops between request and reply? A: Pyro4 raises a `CommunicationError`. The client must decide whether to retry. Note: factorial is idempotent (`fact(5)` always returns `120`) so retrying is safe — but for general operations like "transfer money", you'd need idempotency keys to prevent double execution.

Practical 2 — Java RMI: String Concatenation

2.1 Aim

A distributed application using **Java RMI**: client sends strings; server returns the concatenated/transformed result.

2.2 Real-world analogy

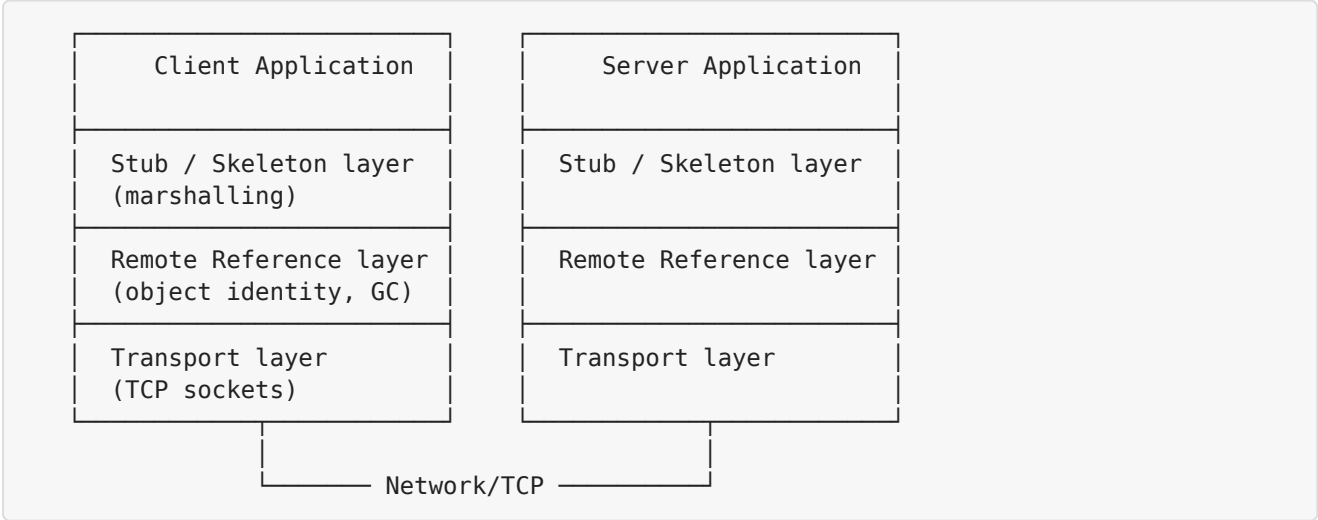
A **library inter-loan**. Your local library doesn't have the book you want, so they call the central library. The central library has the book, processes the request, and the book is sent to you. You only ever interact with your local librarian — the central library is *remote* but accessible *as if it were local* through the librarian (the stub).

2.3 Theory

2.3.1 What is RMI?

Remote Method Invocation is Java's native RPC mechanism. Two big differences from generic RPC: 1. Calls are **method calls on remote objects** (not just functions). 2. You can **pass entire objects** as arguments, not just primitive values — Java's Object Serialization handles it.

2.3.2 RMI Architecture — the three layers



2.3.3 Key Java classes

Class / Interface	Purpose
`java.rmi.Remote`	Marker interface every remotable interface must extend
`java.rmi.RemoteException`	Mandatory checked exception for every remote method
`java.rmi.server.UnicastRemoteObject`	Superclass that turns an object into a remote object — sets up sockets, generates the stub
`java.rmi.registry.LocateRegistry`	Static methods to create or look up the RMI registry
`java.rmi.registry.Registry`	The registry itself: `bind`, `rebind`, `lookup`, `unbind`, `list`

2.3.4 The RMI Registry

A small daemon that holds `name → remote object stub` mappings. Default port **1099**. Think of it as a phone book.

How to start it: 1. Run `rmiregistry` from the command line, **or** 2. **Embed it inside your server JVM** with `LocateRegistry.createRegistry(1099)`. This is what we do because the external command is finicky about the classpath.

2.3.5 `bind` vs `rebind`

- `bind(name, obj)` — fails with `AlreadyBoundException` if the name is already taken.
- `rebind(name, obj)` — silently replaces. Use this so you can restart the server cleanly.

2.3.6 Pass-by-value vs Pass-by-reference

- **Local objects** (`Serializable` but not `Remote`) are passed **by value** — a *copy* is sent. *Example:* a `String` argument.
- **Remote objects** (extend `UnicastRemoteObject`) are passed **by reference** — only the stub goes; the actual object stays on the server.

This means if you pass a `Hotel` remote object to another machine, that machine can call methods back on the original server.

2.3.7 Java Object Serialization

Built into the JDK. Any class implementing `java.io.Serializable` can be serialized to a byte stream. RMI uses this internally for marshalling. Strings, primitives, and most JDK collections are already Serializable.

2.4 Worked example — Tracing `obj.concat("Hello", "World")`

```
Client                                                  Server
------                                                  ------
1. Lookup "concat" in registry on port 1099.
2. Get back a stub bound to host:port of server.

3. Call obj.concat("Hello", "World")
4. Stub marshals:
     { method:"concat", args:["Hello","World"] }
   ——— bytes over TCP ———>
                                            5. Skeleton unmarshals.
                                            6. Calls Server.concat("Hello","World")
                                                   returns "HelloWorld"
                                            7. Skeleton marshals "HelloWorld"
                                                <——— bytes back ———
8. Stub unmarshals "HelloWorld".
9. Returns to caller; client prints it.
```

2.5 Why Java for RMI?

- **Built into the JDK** — `java.rmi.*`. No third-party libraries.
- **Cross-platform bytecode** — compile once, run anywhere.
- **Object Serialization** is also built in — pass any `Serializable` object as easily as a primitive.
- **Type safety** — the `StringConcat` interface enforces method signatures at compile time. A typo in a method name fails to compile, not at runtime.
- **Compared to Python's Pyro4**: Java has stronger compile-time checks; Pyro4 is more dynamic.
- **Compared to gRPC**: no `.proto` IDL file to maintain — the Java interface itself is the contract.

2.6 Code walkthrough

`StringConcat.java`

```java
public interface StringConcat extends Remote {
    String concat(String a, String b) throws RemoteException;
    ...
}
```

- `extends Remote` — without it, RMI ignores the interface.
- `throws RemoteException` on every method — *mandatory*. Forces the caller to handle network failure.

`Server.java`

```
public class Server extends UnicastRemoteObject implements StringConcat {
    Server() throws RemoteException {}
```

- `extends UnicastRemoteObject` — tells RMI to export this object: open a server socket, generate a stub, register with the RMI runtime.
- The constructor must declare `RemoteException` because the superclass does.

```
Registry registry = LocateRegistry.createRegistry(1099);
registry.rebind("concat", obj);
```

- `createRegistry(1099)` starts a registry **inside this JVM**. Now `rmiregistry` does not need to be running separately.
- `rebind("concat", obj)` registers the object under the name “concat”, so clients can find it via `lookup("concat")`.

`Client.java`

```
Registry registry = LocateRegistry.getRegistry("localhost", 1099);
StringConcat obj = (StringConcat) registry.lookup("concat");
```

- Connects to the registry.
- `lookup("concat")` returns the stub.
- Cast to the *interface* — the client never knows the implementation class.

```
obj.concat("Hello", "World")
```

Goes through the stub: marshal → network → server → execute → marshal → network → unmarshal → return.

2.7 Where RMI / similar tech is used today

- **Java EE / Jakarta EE** — Enterprise JavaBeans (EJB) use RMI under the hood.
- **JMX** (Java Management Extensions) — uses RMI for remote monitoring.
- **JBoss / WildFly** application servers — RMI for clustering.
- **Eclipse JDT** — uses RMI variants between debugger and target VM.
- **Modern alternatives** that have largely replaced RMI: **gRPC**, **REST APIs**, **GraphQL**.

2.8 Likely viva questions

Q: What is RMI? A: Java’s mechanism for invoking methods on objects in another JVM, often on another host. Built on TCP and Java Object Serialization.

Q: How is RMI different from RPC? A: | | RPC | RMI | |-|—-|—-| | Paradigm | Procedural | Object-oriented | | Languages | Language-neutral | Java only | | Marshalling | XDR/protobuf/JSON | Java Object Serialization | | Pass-by | Value | Value (Serializable),

Reference (Remote) | | Stubs | Generated by IDL compiler | Generated automatically since Java 1.5 |

Q: Why must a remote interface extend `Remote`? A: It signals to the RMI runtime that this interface defines remotely-callable methods, and triggers stub generation. Without it the methods are just normal Java methods.

Q: Why must every remote method declare `RemoteException`? A: To force callers to handle network failures (host unreachable, marshalling failure, timeout). It's a checked exception — compilation fails if you don't handle it.

Q: What is `UnicastRemoteObject`? A: A superclass that, when extended, exports the object: opens a server socket, generates a stub, registers with the RMI runtime. "Unicast" means a single TCP connection (vs multicast).

Q: What port does the RMI registry run on? A: 1099 by default.

Q: Difference between bind and rebind? A: `bind` fails if the name is already taken; `rebind` silently overwrites.

Q: How are arguments passed? A: Local Serializable objects are passed by value (copy sent over the wire). Remote objects are passed by reference (only the stub travels; calls go back to the original server).

Q: What is a stub? A: A client-side proxy implementing the same interface as the remote object. It marshals arguments, sends them, blocks for the reply, unmarshals it, and returns to the caller — making the remote call look local.

Q: How does RMI achieve at-most-once semantics? A: TCP guarantees ordered, reliable delivery. The JVM throws `RemoteException` on any failure rather than silently retrying — preserving the at-most-once guarantee.

Q: Why doesn't your code use a separate `rmiregistry` process? A: Because the external `rmiregistry` command is sensitive to classpath setup. Calling `LocateRegistry.createRegistry(1099)` inside the server JVM avoids that headache.

Practical 3 — MapReduce Word Count on Hadoop

3.1 Aim

Count occurrences of each word in a text file using **Hadoop MapReduce** across a (pseudo-)cluster.

3.2 Real-world analogy

A national election count.

- **Map phase:** Each polling booth counts the votes cast at it locally and produces tally sheets like `(BJP, 743), (Congress, 412), (NOTA, 18)`.
- **Shuffle phase:** All `BJP` tally sheets are sent to one central counter, all `Congress` sheets to another, and so on.
- **Reduce phase:** Each central counter sums their sheets to produce the final national total per party.

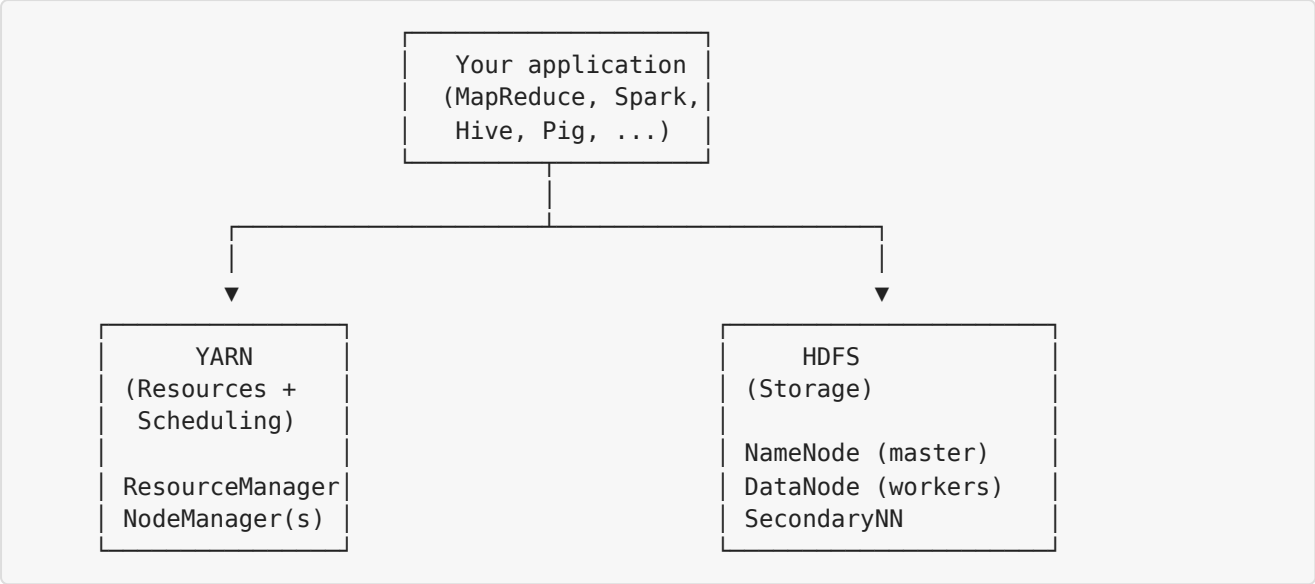
That's exactly MapReduce. Google invented this pattern (paper, 2004) to index the web — so you can think of "word count" as a tiny version of "build a search index over every web page".

3.3 Theory

3.3.1 Why do we need this?

A single machine can't store or process terabytes/petabytes of data. So: - **Spread the data** across many cheap machines. - **Move computation to where the data lives** (don't ship terabytes over the network). - **Tolerate failures** — when a machine dies, redo just its tasks.

3.3.2 The Hadoop ecosystem



3.3.3 HDFS — Hadoop Distributed File System

- Files are split into **128 MB blocks**, each stored on multiple DataNodes (default replication = 3).
- **NameNode** is the master — keeps the metadata (which file has which blocks on which DataNodes).
- **DataNodes** hold the actual blocks. They send heartbeats to the NameNode every 3 seconds; if a DataNode misses many heartbeats, the NameNode treats it as dead and replicates its blocks elsewhere.
- **SecondaryNameNode** is *not* a hot backup. It periodically merges the NameNode's edit log with its in-memory state to keep startup time bounded.

Why 128 MB blocks? Big blocks minimise NameNode metadata (fewer blocks to track), maximise sequential disk read throughput, and match analytics workloads (read whole files, rarely random access).

3.3.4 YARN — Yet Another Resource Negotiator

Component	Job
ResourceManager (RM)	Cluster-wide scheduler; allocates resources
NodeManager (NM)	Runs on every worker; manages containers on that node
ApplicationMaster (AM)	One per running application; negotiates resources with RM
Container	A bundle of CPU + RAM running one task

Default ports: - RM client API: **8032** - RM web UI: **8088** - NameNode RPC: **9000** - NameNode web UI: **9870**

3.3.5 MapReduce model

A computation expressed as two functions:

```
map(k1, v1)    -> list of (k2, v2)
reduce(k2, list of v2) -> list of (k3, v3)
```

Between Map and Reduce, Hadoop performs **shuffle and sort** automatically: it groups all values with the same key and ships them to the reducer responsible for that key.

3.3.6 Word Count traced through MapReduce

Suppose `input.txt`:

```
hello world hello hadoop
mapreduce is fun hadoop is powerful
hello mapreduce world
```

Map phase (each line is processed independently):

```
Line 1 -> ("hello",1) ("world",1) ("hello",1) ("hadoop",1)
Line 2 -> ("mapreduce",1) ("is",1) ("fun",1) ("hadoop",1) ("is",1) ("powerful",1)
Line 3 -> ("hello",1) ("mapreduce",1) ("world",1)
```

Shuffle and sort (Hadoop groups by key):

```
"fun"       -> [1]
"hadoop"    -> [1, 1]
"hello"     -> [1, 1, 1]
"is"        -> [1, 1]
"mapreduce" -> [1, 1]
"powerful"  -> [1]
"world"     -> [1, 1]
```

Reduce phase (sum the lists):

```
("fun",1) ("hadoop",2) ("hello",3) ("is",2) ("mapreduce",2) ("powerful",1) ("world",2)
```

That's exactly what your job produces.

3.3.7 Other key MapReduce concepts

- **Combiner** — an optional mini-reducer that runs on the *map machine* before shuffle. For word count, sums local counts so we send `("hello", 2)` instead of `("hello",1)` `("hello",1)`. Reduces network traffic. Must be associative + commutative.
- **Partitioner** — decides which reducer a key goes to. Default: `hash(key) % numReducers`.
- **InputFormat / RecordReader** — how a file becomes (k, v) pairs. `TextInputFormat` (default) gives `(byteOffset, line)`.
- **Speculative execution** — if a task is slow, Hadoop launches a duplicate; first to finish wins. Helps with stragglers.

- **Data locality** — Hadoop schedules a map task on the DataNode holding the input block, avoiding network reads.

3.3.8 Hadoop's Writable types

Hadoop has its own serialization, smaller and faster than Java's. Every key/value crossing the network must be a `Writable`. - `IntWritable` ≈ `int` - `LongWritable` ≈ `long` - `Text` ≈ `String` - `NullWritable` (singleton; no value)

3.3.9 Modes of operation

Mode	Daemons	Use
Standalone (local)	None	Dev / debugging on one machine
Pseudo-distributed	All on one machine	What you have set up
Fully distributed	NameNode + many DataNodes / NodeManagers	Production

3.4 Worked example with the actual job

```
Input:  /input/input.txt   (3 lines, ~83 bytes)
Output: /output/part-r-00000

Job ID: application_1777950509386_0001
Map tasks:    1   (file is < 128 MB, one block, one map task)
Reduce tasks: 1   (default)

Counters:
  Map input records:    3   (3 lines)
  Map output records:   13  (13 word tokens emitted)
  Reduce input groups:  7   (7 distinct words)
  Reduce output records:7   (7 (word,count) pairs)
```

3.5 Why Java for MapReduce?

- **Hadoop is written in Java.** The native MapReduce API is Java; programs run in the same JVM as the Hadoop libraries — zero IPC overhead.
- **Hadoop Streaming** lets you write mappers/reducers in Python or Ruby, but each record must be piped via stdin/stdout to the external script — slower.
- **Type safety** of generics (`Mapper<Object, Text, Text, IntWritable>`) prevents many runtime errors.
- Modern shops often use **Spark with Scala or Python** because it's faster and easier — but the syllabus targets MapReduce specifically.

3.6 Code walkthrough

`WordMapper.java`

```
public class WordMapper extends Mapper<Object, Text, Text, IntWritable> {
```

Generic params: `<INPUT_KEY, INPUT_VALUE, OUTPUT_KEY, OUTPUT_VALUE>`. For `TextInputFormat` (default), input key is byte offset (we ignore it), input value is the line as `Text`.

```
public void map(Object key, Text value, Context context) {
    StringTokenizer st = new StringTokenizer(value.toString());
    while (st.hasMoreTokens()) {
        context.write(new Text(st.nextToken()), new IntWritable(1));
    }
}
```

Splits the line on whitespace, emits `(word, 1)` for every token. `context.write` is how Mappers/Reducers emit output.

`WordReducer.java`

```
public class WordReducer extends Reducer<Text, IntWritable, Text, IntWritable> {
    public void reduce(Text key, Iterable<IntWritable> values, Context context) {
        int sum = 0;
        for (IntWritable val : values) sum += val.get();
        context.write(key, new IntWritable(sum));
    }
}
```

For each unique word (key), the framework calls `reduce` once with **all** its values bundled in `Iterable`. We loop and sum, then emit the total.

`WordCount.java` (driver)

Wires Mapper, Reducer, output types, input/output paths, and submits the job.

3.7 Where MapReduce is used in real life

- **Google** invented it to build their web index.
- **Facebook** processes user-action logs (Hive runs MapReduce under the hood).
- **Yahoo!** historically used MapReduce for search and ads.
- **LinkedIn** processes “people you may know” with MapReduce-like batch jobs.
- **eBay** used it for clickstream analytics.
- **Banks and telecoms** use it for fraud detection, ETL, and regulatory reports.
- **Modern world** has largely replaced MapReduce with **Spark** (10–100× faster for iterative algorithms).

3.8 Likely viva questions

Q: What problem does MapReduce solve? A: Processing huge datasets that don't fit on one machine, in parallel across a cluster, by expressing the computation as two simple functions (`map`, `reduce`) that the framework can automatically distribute, retry, and load-balance.

Q: Walk me through Word Count in MapReduce. A: Input lines → Map emits `(word, 1)` for each token → Shuffle groups by word → Reduce sums the 1s → Output `(word, total)`.

Q: What is the Combiner and why use it? A: A local mini-reducer that runs on the mapper's machine to compress map output before the shuffle. For Word Count, it sums local counts so we transmit `("hello", 5)` instead of five separate 1s. Cuts network traffic dramatically.

Q: What is shuffle and sort? A: The framework's automatic phase between Map and Reduce: partition keys, sort them, and ship all values for each key to the reducer assigned to that key.

Q: What's a partitioner? A: The component that decides which reducer a given key goes to. Default: `hash(key) % numReducers`.

Q: Why is the HDFS block size 128 MB? A: Big blocks minimise NameNode metadata, maximise sequential read throughput, and match analytics workloads (sequential reads of large files).

Q: SecondaryNameNode — is it a hot standby? A: **No.** It only periodically checkpoints the NameNode's metadata. A real high-availability setup uses a Standby NameNode plus ZooKeeper / JournalNodes.

Q: Why use Hadoop's Writable types instead of Java's `int` / `String`? A: Writables have a custom binary format much smaller and faster than Java's default serialization, designed specifically for Hadoop's framework.

Q: What is YARN and what's its role? A: A cluster resource manager. It abstracts CPU/RAM into containers and lets multiple frameworks (MapReduce, Spark, Tez) share the cluster.

Q: What ports do you need open? A: 9000 (NameNode), 9870 (NameNode UI), 8032 (RM), 8088 (RM UI).

Q: What is data locality? A: Hadoop tries to schedule a map task on the same node that holds its input block, avoiding network reads — usually the slowest part of the job.

Q: Difference between Hadoop and Spark? A: Hadoop MapReduce reads/writes intermediate data to disk (slow but fault-tolerant). Spark keeps data in memory (RDDs/DataFrames), making iterative algorithms 10–100× faster. Both can run on YARN.

Practical 4 — Fuzzy Set Operations

4.1 Aim

Implement union, intersection, complement, and difference on **fuzzy sets**.

4.2 Real-world analogy

The temperature of your bath water.

- A *crisp* (classical) view: water is either “hot” (above 40°C) or “not hot” (40°C or below).
  - 39.9°C — not hot
  - 40.0°C — hot ← discontinuous jump
- A *fuzzy* view: water has a degree of hot-ness in [0,1]:
  - 25°C → 0.0 hot
  - 35°C → 0.3 hot
  - 40°C → 0.7 hot
  - 50°C → 1.0 hot

This matches how humans actually think — there’s no sharp line.

4.3 Theory

4.3.1 Crisp vs Fuzzy

- **Crisp set:** every element is either fully in (1) or fully out (0). The world of Boolean logic.
- **Fuzzy set:** elements have a *degree of membership* μ(x) ∈ [0, 1]. Captures vagueness.

4.3.2 Membership function μ(x)

A function that maps each element x in the universe to a value in [0, 1]. Common shapes:

Examples: - “Tall person” — triangular peaking at 195 cm. - “Hot day in Pune” — sigmoid rising sharply from 30°C.

4.3.3 Standard operations (Zadeh)

Operation	Definition
Union (A ∪ B)	μ(x) = **max**(μ_A(x), μ_B(x))
Intersection (A ∩ B)	μ(x) = **min**(μ_A(x), μ_B(x))
Complement (A’)	μ(x) = **1 − μ_A(x)**
Difference (A − B)	μ(x) = min(μ_A(x), 1 − μ_B(x))

4.3.4 Worked example

```
Universe: {x1, x2, x3, x4}
A = [0.2, 0.5, 0.7, 0.9]
B = [0.3, 0.6, 0.4, 0.8]

A ∪ B = [max(0.2,0.3), max(0.5,0.6), max(0.7,0.4), max(0.9,0.8)]
      = [0.3, 0.6, 0.7, 0.9]

A ∩ B = [min(0.2,0.3), min(0.5,0.6), min(0.7,0.4), min(0.9,0.8)]
      = [0.2, 0.5, 0.4, 0.8]

A'    = [1-0.2, 1-0.5, 1-0.7, 1-0.9]
      = [0.8, 0.5, 0.3, 0.1]

A − B = [min(0.2,1-0.3), min(0.5,1-0.6), min(0.7,1-0.4), min(0.9,1-0.8)]
      = [min(0.2,0.7), min(0.5,0.4), min(0.7,0.6), min(0.9,0.2)]
      = [0.2, 0.4, 0.6, 0.2]
```

4.3.5 Properties

- **Idempotent:** A ∪ A = A, A ∩ A = A
- **Commutative:** A ∪ B = B ∪ A
- **Associative:** (A ∪ B) ∪ C = A ∪ (B ∪ C)
- **De Morgan’s laws:** (A ∪ B)’ = A’ ∩ B’
- **Excluded middle FAILS:** A ∪ A’ ≠ universe in general
  - Example: μ_A(x) = 0.5 → μ_A∪A’(x) = max(0.5, 0.5) = 0.5, **not** 1.

4.3.6 Fuzzy logic vs probability

Often confused — they are different things.

	Probability	Fuzzy
What it represents	Likelihood of an event	Degree of membership in a category
Example: “It will rain tomorrow with 0.7”	70% chance, then it rains or doesn’t	— (not a fuzzy statement)
Example: “35°C is hot with degree 0.7”	—	35°C belongs to the category “hot” with degree 0.7
Math	Kolmogorov axioms	Zadeh’s max/min

4.4 Why Python?

- Operations are list comprehensions — Python expresses them almost mathematically.
- No performance need; concept is what matters.
- Could use any language; Python is shortest and clearest.

4.5 Code walkthrough

```
def fuzzy_union(a, b):
    return [max(x, y) for x, y in zip(a, b)]
```

Element-wise `max` — exactly the Zadeh definition.

```
def fuzzy_complement(a):
    return [round(1 - x, 4) for x in a]
```

The `round(..., 4)` cleans up floating-point ugliness like `0.30000000000000004` from `1 - 0.7`.

The interactive shell repeatedly reads input, validates `0 ≤ x ≤ 1`, and lets the user pick an operation from a menu.

4.6 Where fuzzy logic is used in real life

- **Sony, LG, Whirlpool washing machines** — fuzzy rules adjust water level / spin speed by load size & fabric type.
- **Subway brakes** in the Sendai Subway, Japan (since 1987). Fuzzy controllers stop trains 2× more smoothly than classical controllers.
- **Camera autofocus** (Canon, Nikon).

- **Air conditioners and rice cookers** — fuzzy controllers smooth temperature transitions.
- **Anti-lock braking systems (ABS)** in cars.
- **Medical diagnosis systems** — symptoms like “mild fever” are inherently fuzzy.
- **Stock-trading systems** — “buy if RSI is `low` AND momentum is `strong`”.

4.7 Likely viva questions

Q: What is a fuzzy set? A: A set where every element has a degree of membership in [0, 1] — not the binary in/out of classical sets.

Q: What is a membership function? A: A mapping μ : U → [0, 1] giving the membership degree of each element of the universe U.

Q: Define union, intersection, complement. A: Union: max. Intersection: min. Complement: `1 − μ(x)`.

Q: Why does the law of excluded middle fail? A: Because both μ_A(x) and μ_A’(x) = 1 − μ_A(x) are in [0,1]. For μ_A = 0.5, max(0.5, 0.5) = 0.5, not 1. So A ∪ A’ is not the universe.

Q: Real example of fuzzy logic at work? A: Air conditioner: “IF temperature is HIGH AND humidity is HIGH THEN cooling is FAST”. Membership of “HIGH” is fuzzy, allowing smooth control.

Q: Difference between probability and fuzzy? A: Probability is about likelihood of yes/no events. Fuzzy is about degree of belonging to a category. A 0.7 probability of rain means 70% chance rain happens (then it’s binary). A 0.7 membership in “hot” means a temperature *partially* qualifies as hot.

Practical 5 — Load Balancing Simulation

5.1 Aim

Simulate distributing client requests across multiple servers using load-balancing algorithms.

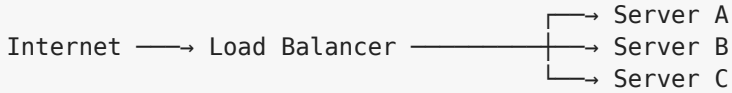
5.2 Real-world analogy

A bank with 5 teller windows. A queue manager directs each customer to a teller using some policy: - **Round Robin** — customer #1 to teller 1, #2 to teller 2, #3 to teller 3 …, #6 back to teller 1. - **Least busy** — send to the teller currently helping the fewest people. - **Random** — pick a teller at random. - **VIP fast lane** — send “premium” customers to the teller with extra training (weighted).

That’s exactly what a load balancer does for web traffic.

5.3 Theory

5.3.1 What does a load balancer do?



Goals: even resource use, higher throughput, lower latency, high availability (route around failed servers), horizontal scalability (add servers anytime).

5.3.2 Layer 4 vs Layer 7

Layer	Looks at	Speed	Capabilities
L4 (transport)	IP + port (TCP/UDP)	Very fast	Just route bytes
L7 (application)	HTTP headers, URL, cookies	Slower	Path-based routing, SSL termination, sticky sessions

5.3.3 Algorithms with worked examples

Round Robin: 10 requests, 3 servers

```
Request: 1 2 3 4 5 6 7 8 9 10
Server : A B C A B C A B C A
Counts : A=4, B=3, C=3
```

Random: Random choice; statistically even after enough requests but uneven for small N.

```
Request: 1 2 3 4 5 6 7 8 9 10
Server : C A B B A C A B B C
Counts : A=3, B=4, C=3   (uneven for 10 requests)
```

Least Connections: Tracks active count; sends new request to the smallest.

```
After  request 1: counts = {A:1, B:0, C:0} -> next goes to B
After  request 2: counts = {A:1, B:1, C:0} -> next goes to C
After  request 3: counts = {A:1, B:1, C:1} -> next goes to A (tie-break)
After  request 4: counts = {A:2, B:1, C:1} -> next goes to B
... very even.
```

Weighted Round Robin: weights A=3, B=1, C=1

```
Expanded sequence: A A A B C
Request:           1 2 3 4 5 6 7 8 9 10
Server:            A A A B C A A A B C
Counts:            A=6, B=2, C=2
```

5.3.4 Health checks

The LB pings each server every few seconds. A typical config: - Try to open a TCP connection on port 80, OR - Send `GET /health` and expect `HTTP 200`.

If a server fails N consecutive checks, it’s removed from rotation. When it recovers (passes M consecutive checks), it’s added back.

5.3.5 Sticky sessions

“Once you’re routed to server X, all your future requests also go to X.” - Implemented via a cookie or by hashing source IP. - Trade-off: simpler in-memory session state, but reduces fairness and breaks if X dies.

5.3.6 Real-world load balancers

Type	Examples
Hardware	F5 BIG-IP, Citrix NetScaler
Software	**Nginx**, **HAProxy**, **Envoy**, Apache Traffic Server
Cloud	**AWS ELB / ALB / NLB**, GCP Load Balancing, Azure LB
CDN-edge	Cloudflare, Akamai

5.4 Why Python?

- Simulating an algorithm needs only a list and a loop — Python expresses it in 3 lines.
- No real network is involved; we're showing the *logic*.
- Real production load balancers are written in C (Nginx) or C++ (Envoy) for speed.

5.5 Code walkthrough

```
def round_robin(servers, n):
    return [servers[i % len(servers)] for i in range(n)]
```

The `%` (mod) cycles through indices 0, 1, …, N-1, 0, 1, …

```
def least_connections(servers, n):
    counts = {s: 0 for s in servers}
    assignments = []
    for _ in range(n):
        target = min(counts, key=counts.get)
        counts[target] += 1
        assignments.append(target)
    return assignments
```

`min(counts, key=counts.get)` finds the key with the smallest value — i.e., the server with fewest active connections.

The bar-chart distribution at the end visually shows fairness.

5.6 Where load balancers are used in real life

- Every site you visit (Google, Facebook, Instagram, Amazon) has a load balancer fronting hundreds or thousands of servers.
- **Netflix** routes 200 million subscribers' streams through a tiered LB system.
- **Uber** load-balances trip requests across regional services.
- **WhatsApp** load-balances 100 billion messages/day.
- Your **college's WiFi captive portal** is probably load-balanced.

5.7 Likely viva questions

Q: Why do we need load balancers? A: To prevent any one server from being overloaded, to scale horizontally, to provide failover, and to give a single endpoint to clients.

Q: Compare Round Robin and Least Connections. A: Round Robin is stateless and simple; works perfectly when servers are identical and requests are uniform. Least Connections tracks active load and adapts to non-uniform request durations (good for long-lived connections like WebSockets or DB pools).

Q: Layer 4 vs Layer 7? A: L4 inspects only IP/port — fast, content-agnostic. L7 inspects HTTP headers/URLs — enables routing rules, SSL termination, but slower.

Q: Sticky sessions — pros and cons? A: Pro: simpler in-memory session state on app servers. Con: uneven distribution, hard to scale, broken when a server dies.

Q: What if the load balancer itself fails? A: That's the single point of failure! Mitigated by deploying multiple LBs behind DNS round robin or using floating virtual IPs (keepalived / VRRP) so a backup LB takes over.

Q: How do AWS ELB / Nginx do health checks? A: Periodic TCP/HTTP probes to `/health`. Unhealthy targets are removed from the pool until they recover.

Q: When would you use weighted round robin? A: When servers have different capacities (e.g., one is a beefy 32-core machine, another is 8-core).

Practical 6 — Clonal Selection Algorithm

6.1 Aim

Implement the **Clonal Selection Algorithm (CSA)** — an immune-system-inspired optimization technique.

6.2 Real-world analogy

How vaccinations work.

When a flu virus enters your body: 1. Specialized B-cells with receptors that *match* the virus are **selected**. 2. They **clone themselves rapidly** (your body makes millions in a few days). 3. As they clone, small **mutations** in the receptor improve the match. 4. The best-fitting clones become **memory cells** — next time the same virus shows up, you're immune.

CSA mimics this: - Virus = the optimization target - B-cell = a candidate solution (antibody) - Match strength = fitness/affinity - Cloning + mutation = generating new candidates near the current best

6.3 Theory

6.3.1 Algorithm steps (CLONALG)

```
1. Initialize random population of antibodies
2. For each generation:
     a. Evaluate affinity (fitness) of each antibody
     b. Select n best
     c. Clone them (better antibodies get more clones — affinity-proportional)
     d. Mutate clones (better antibodies mutate less — hypermutation)
     e. Evaluate clones
     f. Replace worst antibodies with best clones + some random new ones
3. Return best antibody found
```

6.3.2 CSA vs Genetic Algorithm

	CSA	GA
Inspiration	Immune system	Natural selection
Crossover?	No	Yes
Variation	Hypermutation only	Mutation + crossover
Strength	Maintains diversity, multimodal optimization	Faster on unimodal

6.3.3 Affinity

Problem-specific. For minimization, often `affinity = 1 / (1 + f(x))` so smaller f → higher affinity.

6.4 Worked example — find x that minimises |x − 42|

Setup: target = 42, population size 10, n_select = 3, mutation range ±5, 5 generations.

```
GEN 0: random population
       [78, 12, 51, 33, 67, 8, 45, 91, 25, 58]

GEN 1: sort by distance to 42 → [45, 33, 51, 25, 58, 67, 12, 78, 91, 8]
       best = [45, 33, 51]
       clones (with mutation): [45+3=48, 33-2=31, 51+1=52]
       new pop = [45, 33, 51, 48, 31, 52, 25, 58, 67, 12]

GEN 2: sort → [45, 48, 33, 51, 52, 31, 25, 58, 67, 12]
       best = [45, 48, 33]
       clones: [45-1=44, 48+2=50, 33+4=37]
       new pop = [45, 48, 33, 44, 50, 37, 51, 52, 31, 25]

GEN 3: sort → [44, 45, 48, 37, 50, 33, 51, 52, 31, 25]
       best = [44, 45, 48]
       clones: [44+0=44, 45-3=42, 48-2=46]
       new pop = [44, 45, 48, 44, 42, 46, 37, 50, 33, 51]

GEN 4: sort → [42, 44, 44, 45, 46, 48, 37, 50, 33, 51]
       best = [42, 44, 44]
       ... (already converged to 42!)

Best after 5 gens: x = 42, distance = 0
```

The population concentrates around the target each generation.

6.5 Why Python?

- Pure algorithm — no networking, no GUI, no large-data libraries needed.
- `random` and list comprehensions express CSA in ~30 lines.
- Real implementations would use NumPy for vectorization.

6.6 Code walkthrough

```
population = [random.randint(target - 50, target + 50) for _ in range(pop_size)]
```

Random initial antibodies near the target so we don't waste generations exploring far away.

```
population.sort(key=lambda x: affinity(x, target))
best = population[:n_select]
```

Sort ascending by distance (smaller distance = higher affinity). Take the top `n_select`.

```
clones = [b + random.randint(-mutation_range, mutation_range) for b in best]
```

Clone & mutate. In a more faithful CSA, the *number* of clones and the *amount* of mutation depend on the antibody's affinity rank.

```
population = (best + clones)
population.sort(...)
population = population[:pop_size]
```

Combine elites + clones, sort, trim back to pop_size — the classic elitist replacement.

6.7 Where CSA is used in real life

- **Pattern recognition** in handwritten character classification.
- **Network intrusion detection** — antibodies are signatures of "normal" traffic; deviations look like attacks.
- **Job-shop scheduling** in manufacturing.
- **Multimodal function optimization** where GAs get stuck on local optima.
- **Image segmentation** in medical scans.

6.8 Likely viva questions

Q: What is the Clonal Selection Algorithm? A: An optimization heuristic inspired by how B-cells select, clone, and mutate to produce antibodies that match antigens with high affinity.

Q: What is affinity? A: A measure of how well an antibody (candidate solution) matches the antigen (target/fitness goal). Higher affinity = better candidate.

Q: How is CSA different from a Genetic Algorithm? A: CSA uses cloning + hypermutation only — no crossover. CSA tends to maintain diversity better, making it good at multimodal problems.

Q: What is hypermutation? A: Higher-than-normal mutation rate applied to clones to thoroughly explore the neighborhood of high-affinity antibodies.

Q: What’s the role of randomness? A: It enables exploration. Without random initial population and random mutation, the algorithm couldn’t search.

Q: When does it stop? A: After a fixed number of generations OR when no improvement occurs for N generations OR when affinity reaches a threshold.

Q: Real applications? A: Anomaly detection, scheduling, pattern recognition, function optimization.

Practical 7 — AIS Pattern Recognition for Damage Classification

7.1 Aim

Apply **Artificial Immune System** pattern recognition to classify structural sensor readings as damaged or normal.

7.2 Real-world analogy

Airport security profiling. Over years of seeing normal passengers, security staff develop an internal sense of what “normal” looks like — typical luggage, clothing, behavior. When something is *outside* that pattern (an unusual gait, an oddly bulky bag), it’s flagged.

That’s basically negative selection in immunology: **learn what normal looks like; treat deviations as anomalies**.

For structures: - Normal: bridge sensors read [0.05, 0.10, 0.08, ...] daily - Damaged: a crack causes readings like [0.92, 0.85, 0.95, ...] - We train antibodies to recognize each, then classify new readings.

7.3 Theory

7.3.1 Artificial Immune Systems

Computational models inspired by the vertebrate immune system, used for: - Pattern recognition - Anomaly detection - Classification - Optimization

7.3.2 Key AIS algorithms

Algorithm	Idea
Negative Selection	Generate detectors that DON’T match self; they will match non-self (anomalies)
Clonal Selection (CLONALG)	Practical 6
Immune Network Theory	Antibodies stimulate/suppress each other
Danger Theory	Discriminate based on “danger signals” rather than self/non-self
Dendritic Cell Algorithms	Newer AIS approach

7.3.3 AIS terminology mapped to ML

AIS term	ML term
Antigen	Input pattern / data sample
Antibody	Detector / model parameter
Affinity	Similarity (1 / distance)
Self	Normal data
Non-self	Anomaly / damage
Memory cells	Trained classifier

7.3.4 Distance/affinity measures

Measure	Formula	When to use
Euclidean	$\sqrt{\Sigma(x_i - y_i)^2}$	Real-valued data
Manhattan	Σ	$x_i - y_i$
Hamming	Count of bits differing	Binary strings
Cosine	x·y / (	x
Mahalanobis	Accounts for correlations	Statistical data

7.3.5 Structural Health Monitoring (SHM)

The motivating real-world application. Aircraft wings, bridges, dams, and high-rises have sensors (accelerometers, strain gauges, fiber-optic sensors). Continuous data is processed by classifiers (often AIS-based) to flag damage *before* it becomes catastrophic.

7.4 Worked example

Training data:

```
Normal samples:    [0.05, 0.10, 0.08, 0.12]   (label 0)
Damaged samples:   [0.92, 0.85, 0.95, 0.88]   (label 1)
```

After training, antibody pools (with small random mutations):

```
normal_pool   = [0.06, 0.11, 0.07, 0.13, 0.09]  (5 antibodies)
damaged_pool  = [0.91, 0.86, 0.94, 0.89, 0.93]  (5 antibodies)
```

Classify new reading `x = 0.75`:

```
distance to closest normal antibody:  |0.75 - 0.13| = 0.62
distance to closest damaged antibody: |0.75 - 0.86| = 0.11
0.11 < 0.62  →  DAMAGED
```

Classify `x = 0.20`:

```
distance to closest normal antibody:  |0.20 - 0.13| = 0.07
distance to closest damaged antibody: |0.20 - 0.86| = 0.66
0.07 < 0.66  →  NORMAL
```

This is essentially **1-Nearest-Neighbor classification** with the antibody pool as the training set.

7.5 Why Python?

Same reasoning as Practical 6: pedagogy, brevity, ecosystem (numpy, scipy, scikit-learn). Real industrial SHM systems often combine AIS with neural networks in production.

7.6 Code walkthrough

```python
def train(samples, n_antibodies_per_class=5, mutation_range=0.05):
    normals  = [r for r, lab in samples if lab == 0]
    damaged  = [r for r, lab in samples if lab == 1]

    def grow(seed_pool):
        if not seed_pool:
            return []
        return [random.choice(seed_pool) + random.uniform(-mutation_range,
        mutation_range)
                for _ in range(n_antibodies_per_class)]

    return grow(normals), grow(damaged)
```

For each class, randomly draw labelled examples and slightly perturb them — a tiny clonal-selection-style training step.

```python
def classify(reading, normal_pool, damaged_pool):
    def best_distance(pool):
        return min(abs(reading - ab) for ab in pool) if pool else float("inf")
    return "NORMAL" if best_distance(normal_pool) < best_distance(damaged_pool) else
        "DAMAGED"
```

Nearest-antibody classifier — the new reading is assigned to whichever class has the closest matching antibody.

7.7 Where AIS is used in real life

- **Aircraft structural monitoring** — Boeing 787 has thousands of sensors using AIS-style classifiers.
- **Bridge monitoring** — long-span bridges (e.g., Akashi Kaikyō in Japan) use AIS for fatigue detection.
- **Network intrusion detection** — Snort and similar tools use AIS-like rules.

- **Medical imaging** — anomaly detection in MRI/CT scans.
- **Manufacturing QC** — detect defective products on the assembly line.
- **Cybersecurity** — antivirus signatures evolved from AIS concepts.

7.8 Likely viva questions

Q: What is an Artificial Immune System? A: A family of bio-inspired algorithms based on the immune system, used for pattern recognition, classification, optimization, and anomaly detection.

Q: What is Negative Selection? A: An AIS technique that produces detectors which do NOT match training (self) data; the detectors then identify anomalies (non-self) when deployed.

Q: What is affinity in AIS? A: A similarity measure between antigen (input) and antibody (detector). High affinity ⇒ strong recognition.

Q: Why use AIS for damage classification? A: Damage data is often rare (you can't easily get a damaged-aircraft training set). Immune-inspired approaches excel at one-class anomaly detection and can learn online as new sensor data arrives.

Q: How does this relate to k-Nearest-Neighbor? A: Very similar — we classify by the nearest antibody (k=1). The difference: AIS produces a *compressed* set of antibodies via cloning/mutation rather than storing all training data.

Q: How would you compute affinity for binary data? A: Hamming distance (count of bits differing), then convert to similarity: `1 – distance/length`.

Q: Pros and cons of AIS vs neural networks? A: Pro: easier to interpret, naturally handles one-class problems, online learning is straightforward. Con: slower to converge, less accurate on large labelled datasets, fewer specialized tools/libraries.

Practical 8 — Distributed Evolutionary Algorithm (DEAP)

8.1 Aim

Implement an **evolutionary algorithm** that evolves a population of candidate solutions toward an optimum.

8.2 Real-world analogy

Selective breeding of cows for milk yield.

A farmer has 100 cows. Each season: 1. Measure milk yield (fitness). 2. Pick the top 20 producers as breeders (selection). 3. Mate them — calves inherit traits from both parents (crossover). 4. Occasional random genetic variation (mutation). 5. Next season, the average yield is higher.

After 50 seasons, your herd's milk yield could double. That's exactly an evolutionary algorithm.

8.3 Theory

8.3.1 Family of evolutionary algorithms

Acronym	Stands for	Specialty
GA	Genetic Algorithm	Bit/string chromosomes; crossover dominant
ES	Evolution Strategies	Real-valued vectors; mutation dominant
GP	Genetic Programming	Evolves trees of code
DE	Differential Evolution	Mutation by vector differences
EP	Evolutionary Programming	Originally for finite-state machines

8.3.2 Generic algorithm

```
Initialize random population
Evaluate fitness of each individual
Repeat for N generations:
  Select parents (tournament/roulette/rank)
  Apply crossover  -> children
  Apply mutation
  Evaluate children's fitness
  Replace old population (often with elitism)
Return best individual found
```

8.3.3 Selection methods

- **Tournament (size k)**: pick k random individuals, the best of them wins. Simple and effective. (We use k=3.)
- **Roulette wheel**: probability of selection ∝ fitness. Beware of one super-fit individual dominating.
- **Rank**: probability ∝ rank, not raw fitness. More stable.

8.3.4 Crossover types

Type	How
Single-point	Pick a random point; child = parent1[:p] + parent2[p:]
Two-point	Two split points
Uniform	Each gene chosen from one parent at random
Arithmetic	For numbers: child = (p1 + p2) / 2 (we use this)
SBX	Simulated Binary Crossover for real values

8.3.5 Mutation

Random perturbation. For integers: add ±k. For bit strings: flip each bit with probability p_m. Maintains population diversity and escapes local optima.

8.3.6 Elitism

Carry the best k individuals into the next generation **unchanged**. Prevents loss of the best solution to bad luck.

8.3.7 Convergence and premature convergence

- **Convergence**: population narrows around the optimum.
- **Premature convergence**: population converges on a *local* optimum because diversity collapsed too soon.

Combat with: higher mutation rate, larger population, niching, occasional random restarts.

8.3.8 DEAP — the framework

Distributed Evolutionary Algorithms in Python. Provides: - `creator` for custom Fitness/Individual classes. - `base.Toolbox` to register operators (select, mate, mutate, evaluate). - Built-in algorithms (`eaSimple`, `eaMuPlusLambda`). - Statistics, hall-of-fame, multiprocessing support.

You don't need DEAP to *understand* GA — but DEAP makes large-scale and **distributed** GAs trivial: fitness evaluations can run in parallel across CPU cores or across machines via SCOOP.

8.3.9 Why "distributed"?

Fitness evaluation is **embarrassingly parallel** — each individual is independent. Distributing across N cores gives roughly N× speedup. Critical when fitness evaluation is expensive (training a neural network, running a simulation).

8.4 Worked example — minimize f(x) = (x − 7)² + 3

The function has a global minimum at x=7 with value 3.

```
Domain: [-100, 100]
Population: 8
Generations: 5
Elite: 1
Mutation rate: 0.5
Mutation range: ±5

GEN 0 (random): [-87, 56, -12, 91, 23, -45, 60, 5]
              fitness:  [8839, 2404, 364, 7059, 259, 2707, 2812, 7]
              sorted:   [5, 23, -12, -45, 60, 56, 91, -87]

GEN 1: elite = [5]
       crossover/mutate:
         (5+23)/2 = 14 + 0   = 14
         (23+5)/2 = 14 + 3   = 17
         (-12+60)/2 = 24 - 2 = 22
         (-45+91)/2 = 23 + 1 = 24
         ...
       new population: [5, 14, 17, 22, 24, 9, 11, 6]
       fitness:        [7,52,103,228,292, 7, 19, 4]   (best=4 at x=6)

GEN 2: elite = [6]
       new ones cluster around 6 ± a few:
       new pop: [6, 7, 8, 9, 5, 11, 4, 10]
       fitness: [4, 3, 4, 7, 7, 19, 12, 12]   (best=3 at x=7)  ← FOUND!

GEN 3-5: refines, stays at x=7.
```

In 2 generations the GA found the exact optimum.

8.5 Why Python (and DEAP)?

- DEAP is **the** Python EC library — pure Python, easy install.
- Python's first-class functions make it natural to express EAs (operators are passed around as objects).
- Performance is enough for textbook problems.
- Could use C++ (GAlib), Java (ECJ, Watchmaker), but Python's ecosystem (NumPy, scikit-learn) wins for prototyping.

8.6 Code walkthrough

```
def evolve(domain, pop_size, generations, mutation_rate, mutation_range,
           elite_size, fitness_fn, mode="min", verbose=True):
```

A flexible evolutionary loop driven by user parameters.

```
new_pop = population[:elite_size]              # elitism
while len(new_pop) < pop_size:
    parents = random.sample(population, 3)     # tournament size 3
    parent  = sort_pop(parents)[0]
    other   = random.choice(population)
    child   = (parent + other) // 2            # arithmetic crossover
    if random.random() < mutation_rate:
        child += random.randint(-mutation_range, mutation_range)
        child = max(lo, min(hi, child))        # clamp to domain
    new_pop.append(child)
```

Each generation: keep elites, build offspring via tournament + arithmetic crossover + occasional mutation.

8.7 Where evolutionary algorithms are used in real life

- **NASA antenna design** — the ST5 spacecraft's antenna was designed by a GA (looks like a paperclip; no human would have come up with it).
- **Drug discovery** — evolve molecular structures for binding affinity.
- **Neural network hyperparameter tuning** — Google's AutoML uses evolutionary search.
- **Game AI** — evolve strategies (StarCraft, Pac-Man).
- **Engineering design** — Boeing has used GAs for wing shapes.
- **Financial trading** — evolve trading strategies.
- **Routing problems** — vehicle routing, network optimization.

8.8 Likely viva questions

Q: What is an evolutionary algorithm? A: A population-based optimization technique inspired by biological evolution, using selection, crossover, and mutation across generations

to improve candidate solutions.

Q: Role of crossover vs mutation? A: Crossover combines existing good traits (exploitation); mutation introduces novelty (exploration). Both are needed — crossover alone leads to stagnation; mutation alone is just random search.

Q: What is elitism? A: Keeping the best k individuals unchanged each generation to prevent regression. Without elitism, the best solution can be lost to bad luck in selection or crossover.

Q: How do you avoid premature convergence? A: Higher mutation rate, larger population, niching (penalize crowding), random restarts, diversity preservation operators.

Q: What is a fitness function? A: A function scoring how good a candidate solution is. Drives selection. Must be cheap because it's evaluated thousands of times.

Q: Why "distributed"? A: Fitness evaluations are independent and can be parallelized across CPU cores or machines. DEAP supports `multiprocessing.Pool` and SCOOP for almost-linear speedup.

Q: Difference between GA and CSA? A: GA uses crossover + mutation (sexual reproduction); CSA uses cloning + hypermutation only (asexual immune-cell proliferation). GAs are usually faster on unimodal problems; CSA preserves diversity better.

Q: Tournament selection — why is it popular? A: O(1) cost per selection (no sort needed), easy to tune (just change tournament size), works well in practice.

Practical 9 — Distributed Hotel Booking via Java RMI

9.1 Aim

A distributed hotel booking system using Java RMI: clients book/cancel rooms; server maintains state.

9.2 Real-world analogy

A hotel reception desk. Multiple guests can walk up at the same time — but the receptionist serves them **one at a time**, looking at the master register, verifying availability, writing in the booking, then serving the next guest. Without that one-at-a-time discipline, two guests could simultaneously be told “yes, room 5 is free” and then both end up booked into the same room.

9.3 Theory

All RMI fundamentals from Practical 2 apply. The new concern here is **stateful concurrency**.

9.3.1 Server-side state

The hotel server maintains a `Map<roomNumber, guestName>`. This state must: - **Persist between calls** — kept as an instance field. - **Be thread-safe** — multiple clients may call concurrently.

9.3.2 Concurrency in RMI

Java RMI **dispatches each incoming call in its own thread** by default. Without synchronization, two clients could pass the “is the room free?” check simultaneously and both succeed in booking the same room. Classic **race condition**.

9.3.3 Race condition example

Without synchronization, here’s what could happen:

```
Time   Thread 1 (Alice)           Thread 2 (Bob)
----   ----------------           --------------
T=0    bookRoom(5,"Alice")
T=1    Check: is room 5 free?     bookRoom(5,"Bob")
T=2                                Check: is room 5 free?
T=3    Yes → reserve for Alice
T=4                                Yes → reserve for Bob
T=5    bookings[5] = "Alice"
T=6                                bookings[5] = "Bob"   ← OVERWRITES Alice!

Final state: room 5 is booked by Bob; Alice thinks she has it but doesn't.
```

This is **the** race condition every student should understand.

9.3.4 Fix: synchronization

```
public synchronized String bookRoom(int roomNo, String guest) {
    if (roomNo < 1 || roomNo > TOTAL_ROOMS) return "FAIL: invalid room";
    if (bookings.containsKey(roomNo)) return "FAIL: already booked";
    bookings.put(roomNo, guest);
    return "OK: room booked";
}
```

The `synchronized` keyword acquires a lock on `this`. Only one thread can hold the lock at a time. Now:

```
Time   Thread 1 (Alice)              Thread 2 (Bob)
----   --------------------          --------------
T=0    bookRoom(5,"Alice") - LOCK
T=1    Check: free? Yes              bookRoom(5,"Bob") - WAIT (lock held)
T=2    Reserve → "Alice"             WAIT
T=3    Return OK - UNLOCK
T=4                                   Acquire LOCK
T=5                                   Check: free? NO → return FAIL
T=6                                   UNLOCK

Result: Alice has room 5; Bob got "FAIL: already booked".  CORRECT.
```

9.3.5 Concurrency trade-offs

`synchronized` is a coarse-grained lock — only one thread in any synchronized method at a time. For higher concurrency: - **`ConcurrentHashMap`** for the bookings map (fine-grained locks). - **`ReadWriteLock`** to allow many concurrent reads. - **Optimistic concurrency** (CAS).

For 10 rooms it doesn’t matter; for a real hotel chain handling 100,000 transactions/sec, it would.

9.3.6 Idempotency

`bookRoom(5, "Alice")` is **not idempotent** — calling it twice produces different results (first OK, second FAIL). Same for cancel. This is why at-most-once semantics matter for these calls — a retry could fail unexpectedly or worse.

For idempotent operations (e.g., `setRoomTo("Alice")`), retries are safe.

9.3.7 What if the server crashes?

All bookings are lost — they live only in memory. Real systems persist to a database (Postgres, MySQL, MongoDB) and run multiple replicas behind a load balancer.

9.4 Why Java RMI for this?

- Already covered in P2: built-in, type-safe, object-oriented.
- Stateful distributed objects are RMI's natural model.
- For a *real* hotel booking service you'd build a REST/gRPC microservice on top of a database — but this practical is about showing **how distributed objects with state work**.

9.5 Code walkthrough

`Hotel.java` (interface)

6 methods: `listAvailable`, `listBookings`, `bookRoom`, `cancelRoom`, `roomStatus`, `totalRooms`. The contract.

`HotelServer.java`

```
private final Map<Integer, String> bookings = new LinkedHashMap<>();
```

`LinkedHashMap` preserves **insertion order** — bookings are listed in the order they happened, not arbitrary hash order. Better UX.

```
@Override
public synchronized String bookRoom(int roomNo, String guest) {
    if (roomNo < 1 || roomNo > TOTAL_ROOMS) return "FAIL: room ...";
    if (bookings.containsKey(roomNo))       return "FAIL: already booked ...";
    bookings.put(roomNo, guest.trim());
    return "OK: room ...";
}
```

- `synchronized` provides the atomic check-then-set required.
- Validates inputs to prevent garbage state.
- Returns a status string (OK/FAIL prefix). A more elegant API would throw a custom exception.

`HotelClient.java`

A 6-option menu. Each menu choice corresponds to one RMI call. The client is otherwise stateless — the server is the system of record.

9.6 Where this kind of system is used in real life

- **Airbnb** — millions of properties, billions of bookings. Behind the scenes: stateful microservices, distributed database, eventual consistency.
- **Booking.com** — uses sharded databases + service-mesh RPC.
- **OYO** — runs on similar architecture.
- **Trip.com / MakeMyTrip / Goibibo** — same model.
- **Airline reservations (Sabre, Amadeus)** — historically used CICS + IBM mainframes; modern systems use distributed services.
- **Movie ticket bookings (BookMyShow)** — exact same race-condition challenges with seat reservations.

9.7 Likely viva questions

Q: How does the server maintain state? A: Through instance variables of the `HotelServer` object, which lives for the lifetime of the JVM.

Q: How do you handle concurrency? A: RMI dispatches each call in its own thread. We mark every server method `synchronized`, so only one thread runs at a time. This prevents race conditions like double-booking.

Q: What is a race condition? A: A bug where the outcome depends on the relative timing of concurrent operations. In our hotel: two threads simultaneously check if room 5 is free, both see "yes", and both book it.

Q: What happens if the server crashes? A: All bookings are lost — in-memory only. A production system persists to a database and replicates the server.

Q: Could we build this as a REST API? A: Yes — that's what most real systems do. Trade-off: REST is language-neutral and stateless; RMI is Java-only but more natural for stateful object-oriented services.

Q: Why `LinkedHashMap` instead of `HashMap`? A: To preserve booking insertion order for stable, predictable client output.

Q: What is at-most-once semantics and why is it relevant here? A: Each remote call executes 0 or 1 times. Important because `bookRoom` is non-idempotent — we don't want duplicate bookings due to retries.

Q: How would you scale this to a real hotel chain? A: Replace the in-memory map with a database (Postgres). Run multiple stateless app servers behind a load balancer. Add a Redis cache. For multi-region, use distributed transactions or eventual consistency with conflict resolution.

Q: Why use `synchronized` instead of `ConcurrentHashMap`? A: For 10 rooms, the difference is negligible. For high-traffic systems, `ConcurrentHashMap` allows much more concurrent reads and finer-grained locks, but the logic is harder to get right.

Practical 10 — Ant Colony Optimization for TSP

10.1 Aim

Apply **Ant Colony Optimization (ACO)** to the **Travelling Salesman Problem (TSP)**.

10.2 Real-world analogy

Real ants finding sugar in your kitchen.

You leave a sugar spill on the counter. Within minutes a trail of ants is leading from their nest to the sugar. How did they find the *shortest* path between nest and sugar without a map?

1. The first ants wander randomly. Each leaves a faint **pheromone trail** as it walks.
2. An ant that finds sugar walks back, **reinforcing its pheromone trail**.
3. New ants are biased toward stronger pheromone trails — the more pheromone, the more attractive.
4. Shorter paths get more round-trips per minute → more pheromone deposits → exponentially more attractive.
5. Pheromone evaporates → weak trails fade → only the strongest survives.

Result: in minutes, the colony converges on the shortest route. **No central planner — pure decentralized intelligence.**

ACO mimics this for any graph-routing problem.

10.3 Theory

10.3.1 Travelling Salesman Problem (TSP)

> Given N cities and pairwise distances, find the shortest route that visits every city exactly once and returns to start.

Properties: - **NP-hard** — exact solution is O(N!) (or $O(N^2 \cdot 2^N)$ with Held-Karp dynamic programming). - For N=20: 20! ≈ 2.4×10^{18} permutations — infeasible to enumerate. - For N>~30: we need heuristics.

Heuristic options: nearest neighbor, 2-opt, simulated annealing, GA, ACO.

10.3.2 Swarm intelligence

Algorithms inspired by the collective behavior of decentralized social insects/animals: - ACO (ants) - PSO — Particle Swarm Optimization (birds, fish) - Bee Algorithms - Firefly Algorithm - Cuckoo Search

Common theme: simple agents, local rules, indirect communication, emergent global intelligence.

10.3.3 ACO algorithm (Ant System)

Each iteration:

```
1. Place each ant on a random starting city.
2. Each ant builds a complete tour by probabilistically choosing the next city:
       P(j | current=i) ∝ τ(i,j)^α × η(i,j)^β

   where:
     τ(i,j)  = pheromone on edge (i,j)
     η(i,j)  = 1 / distance(i,j)   (heuristic: short edges preferred)
     α       = pheromone weight
     β       = heuristic weight

3. After all ants finish, evaporate all pheromone:
       τ(i,j) ← (1 − ρ) × τ(i,j)
   where ρ ∈ (0,1) is the evaporation rate.

4. Each ant deposits pheromone on its tour:
       For each edge (i,j) in the ant's tour:
           τ(i,j) += Q / length(tour)

   Shorter tours deposit more pheromone — that's how good paths self-reinforce.

5. Track best tour found across all iterations.
```

10.3.4 Parameter tuning intuition

Parameter	Low value	High value
α	Ants ignore pheromone (random search)	Ants follow herd (premature convergence)
β	Ants ignore distance	Ants behave like nearest-neighbor heuristic
ρ	Pheromone never fades (stagnation)	Trails forgotten too fast (no learning)
Q	Tiny pheromone, slow learning	Huge pheromone, premature convergence

Typical: α=1, β=2-5, ρ=0.5, Q=100, ants ≈ N (cities), iterations 50-200.

10.3.5 ACO variants

- **Ant System (AS)** — original (this practical).
- **Elitist AS** — extra deposit on the best-so-far tour each iteration.

- **Max-Min Ant System (MMAS)** — pheromone bounded in [τ_min, τ_max] to prevent stagnation.
- **Ant Colony System (ACS)** — local pheromone update during tour construction.

10.4 Worked example — 4-city TSP

Cities: A, B, C, D Distance matrix:

```
      A    B    C    D
A     0    2    9   10
B     2    0    6    4
C     9    6    0    8
D    10    4    8    0
```

Optimum tour (by inspection): A → B → D → C → A, length 2+4+8+9 = **23**.

Initial pheromone: τ = 1.0 on every edge. α = 1, β = 2, ρ = 0.5, Q = 100, 3 ants.

Iteration 1: Ant 1 starts at A. Probability of going to each unvisited city:

```
weight(A→B) = 1^1 × (1/2)^2  = 0.25
weight(A→C) = 1^1 × (1/9)^2  = 0.012
weight(A→D) = 1^1 × (1/10)^2 = 0.01
P(A→B) = 0.25 / 0.272 = 0.92
P(A→C) = 0.012 / 0.272 = 0.044
P(A→D) = 0.01 / 0.272 = 0.037
```

Ant 1 picks B (highly likely). At B, choose between C, D:

```
weight(B→C) = 1 × (1/6)^2 = 0.028
weight(B→D) = 1 × (1/4)^2 = 0.063
P(B→D) = 0.063 / 0.091 = 0.69 → likely D.
```

At D, only C remains. Tour: A→B→D→C→A. Length 23.

Suppose Ant 2 ends with tour: A→C→B→D→A. Length 9+6+4+10 = 29. Suppose Ant 3 ends with tour: A→D→C→B→A. Length 10+8+6+2 = 26.

Evaporation: All edges: τ = (1 − 0.5) × 1.0 = 0.5.

Deposit: Each ant deposits Q/length on its edges. - Ant 1 (length 23) deposits 100/23 ≈ 4.35 on A-B, B-D, D-C, C-A. - Ant 2 (length 29) deposits 100/29 ≈ 3.45 on A-C, C-B, B-D, D-A. - Ant 3 (length 26) deposits 100/26 ≈ 3.85 on A-D, D-C, C-B, B-A.

After iteration 1, pheromone matrix:

```
A-B: 0.5 + 4.35 + 3.85 = 8.70
A-C: 0.5 + 4.35 + 3.45 = 8.30
A-D: 0.5 + 3.45 + 3.85 = 7.80
B-D: 0.5 + 4.35 + 3.45 = 8.30
C-D: 0.5 + 4.35 + 3.85 = 8.70
B-C: 0.5 + 3.45 + 3.85 = 7.80
```

The shortest-tour edges (A-B, B-D, D-C) got the largest deposit because that ant had length 23 (shortest). Future ants will favor these edges.

After many iterations, the pheromone on A-B, B-D, D-C dominates → all ants converge on tour 23.

10.5 Why Python?

- ACO is iterative O(N² × ants × iterations) — feasible in Python for tutorial-sized TSPs.
- Random sampling and probabilistic selection are natural in Python.
- Real-world ACO for big TSPs (thousands of cities) uses C++ or NumPy/Cython for speed.

10.6 Code walkthrough

```
def choose_next_city(current, unvisited, pheromone, dist, alpha, beta):
    weights = []
    for j in unvisited:
        tau = pheromone[current][j] ** alpha
        eta = (1.0 / dist[current][j]) ** beta if dist[current][j] > 0 else 0
        weights.append(tau * eta)
    total = sum(weights)
    if total == 0: return random.choice(unvisited)
    r = random.random() * total
    acc = 0
    for j, w in zip(unvisited, weights):
        acc += w
        if acc >= r: return j
    return unvisited[-1]
```

Roulette-wheel selection based on `τ^α · η^β`. The probability of choosing city j is its weight divided by the total.

```
for i in range(n):
    for j in range(n):
        pheromone[i][j] *= (1 - evap)        # evaporation

for tour, length in all_tours:
    deposit = q / length
    for i in range(len(tour) - 1):
        a, b = tour[i], tour[i+1]
        pheromone[a][b] += deposit          # reinforcement
        pheromone[b][a] += deposit
    pheromone[tour[-1]][tour[0]] += deposit
    pheromone[tour[0]][tour[-1]] += deposit
```

Two phases per iteration: evaporation (uniform decay) followed by deposit (each ant lays Q/length, so shorter tours deposit more).

10.7 Where ACO is used in real life

- **UPS / FedEx route planning** — vehicle routing variants of TSP.
- **Network routing** — old TCP/IP papers using AntNet.
- **Telecom switch routing** — British Telecom used ACO for circuit routing.
- **Manufacturing scheduling** — job-shop problems.
- **Bioinformatics** — protein folding, DNA sequence alignment.
- **Image edge detection** — ants traverse pixel intensity gradients.
- **Drone path planning**.

10.8 Likely viva questions

Q: What is TSP and why is it hard? A: Find the shortest tour visiting every city once and returning. NP-hard — no polynomial algorithm known; brute force is O(N!).

Q: How does ACO solve TSP? A: Many "ants" build tours by probabilistic city selection biased by pheromone trails and inverse distance. Pheromone is reinforced on shorter tours and evaporates over time, gradually concentrating on the best path.

Q: What is α and β? A: α weights the pheromone (memory of past tours); β weights the distance heuristic (greed for short edges). High α → ants follow the herd; high β → ants behave like nearest-neighbor.

Q: Why pheromone evaporation? A: Without it, early random tours dominate forever and the search gets stuck. Evaporation lets the colony "forget" weak trails and continue exploring.

Q: ACO vs Genetic Algorithm for TSP? A: ACO is *constructive* (builds tours edge-by-edge using probabilities); GA is *generative* (mutates and crosses whole tours). ACO often performs slightly better on routing-style problems.

Q: What is swarm intelligence? A: Collective problem-solving by decentralized agents following simple local rules — no central controller. Global intelligent behavior emerges from local interactions.

Q: Time complexity per iteration? A: O(ants × N^2) — each ant visits all N cities; each step considers up to N candidate next-cities.

Q: How do you choose number of ants? A: Typically ants ≈ N (number of cities). Too few = under-exploration; too many = wasted compute.

Q: How do you know it's converged? A: Best tour stops improving for many iterations, OR pheromone matrix shows clear "winning" edges with high concentration and others near zero.

Cross-Cutting Viva Questions

These often come at the start or end and test broad understanding across practicals.

Q: Differentiate RPC and RMI. A: | | RPC | RMI | |–|—–|—–| | Paradigm | Procedural | Object-oriented | | Languages | Language-neutral | Java only | | Marshalling | XDR / protobuf / JSON | Java Object Serialization | | Pass-by | Value | Value (Serializable), Reference (Remote) |

Q: Differentiate Hadoop MapReduce and Spark. A: Hadoop MR reads/writes intermediate results to HDFS — fault-tolerant but slow. Spark keeps data in memory (RDDs/DataFrames) — 10–100× faster for iterative algorithms. Both can run on YARN.

Q: What does "distributed" mean in your practicals? A: Components running in separate processes (often on separate machines), coordinating via message passing — TCP sockets in P1, P2, P9; HDFS+YARN in P3; conceptual simulation in P5.

Q: Difference between fuzzy logic and probability? A: Probability is about *likelihood* of yes/no events; fuzzy is about *degrees* of belonging to vague categories. Different mathematics, different intent.

Q: Why are bio-inspired algorithms (CSA, GA, ACO, AIS) useful? A: Many real-world problems are NP-hard or non-differentiable; classical exact methods fail. Bio-inspired algorithms are general-purpose, derivative-free, robust to noise, and find good (not provably optimal) solutions in reasonable time.

Q: Exploration vs exploitation — explain. A: - **Exploration**: search the solution space widely (high mutation, randomness). Avoids getting stuck. - **Exploitation**: refine known good areas (greedy selection, low mutation). Improves quality.

Good optimization needs both. Start by exploring, gradually shift to exploiting. This trade-off is fundamental to all heuristics in P6, P7, P8, P10.

Q: Why do we use pseudo-distributed Hadoop in the lab? A: To exercise the *real* distributed pipeline — NameNode + DataNodes for HDFS, ResourceManager + NodeManagers for YARN — even if all on one machine. Standalone runs in a single JVM, missing the architecture.

Q: Common failure modes in distributed systems? A: - Network partition (split brain) - Node crash - Message loss - Message duplication - Message reordering - Byzantine faults (malicious or buggy nodes) - Clock skew

Q: What is the CAP theorem? A: In a distributed data store you can guarantee at most two of: - **Consistency** — all nodes see the same data - **Availability** — every request gets a response - **Partition tolerance** — system keeps working despite network splits

You always must choose P; the real trade-off is C vs A.

Q: At-most-once vs exactly-once semantics? A: At-most-once = call executes 0 or 1 times (RMI default). Exactly-once is impossible in the presence of arbitrary failures but can be approximated with idempotency keys + retries + server-side deduplication.

Q: Why does fault tolerance matter? A: At scale, failures are *constant* — Google estimates ~1 disk failure per hour at their scale. Systems must keep running through failures. Replication (HDFS), retries (RMI), heartbeats (Hadoop), leader election (ZooKeeper) — all serve this goal.

Q: Synchronous vs asynchronous in practical terms? A: Synchronous: client blocks waiting (RMI, RPC, simple HTTP). Easy to reason about, slow under network latency. Async: client fires and continues (message queues, async/await). More efficient but harder to debug.

Glossary

Term	Meaning
Affinity	Similarity score in immune/CSA algorithms
Antibody	A detector / candidate solution in AIS
Antigen	Input pattern / data sample in AIS
ApplicationMaster	Per-application coordinator in YARN
At-most-once	Call executes 0 or 1 times
Block (HDFS)	128 MB chunk of an HDFS file
Combiner	Map-side mini-reducer to compress shuffle data
Container (YARN)	Resource bundle (CPU+RAM) for a task
Convergence	Population narrowing around the optimum
Crisp set	Classical Boolean set
Crossover	GA operator combining two parents
DataNode	HDFS worker storing blocks
Daemon	Background process listening for requests
DEAP	Distributed Evolutionary Algorithms in Python
Elitism	Keeping best individuals across GA generations
Evaporation (ACO)	Decay of pheromone over time
Fitness	Score driving selection in EAs
Fuzzy set	Set with [0,1] membership values
HDFS	Hadoop Distributed File System
Hypermutation	Higher-than-normal mutation rate on clones (CSA)
Idempotent	Calling twice has the same effect as once
Marshalling	Serializing objects to bytes
Membership function	$\mu(x) \in [0,1]$
MapReduce	Two-phase parallel data processing model
NameNode	HDFS master holding metadata
NodeManager	YARN per-node agent
NP-hard	Class of problems with no known polynomial algorithm
Pheromone (ACO)	Reinforcement value on graph edges
Proxy / Stub	Client-side surrogate for a remote object
Pyro4	Python Remote Objects framework
Race condition	Bug where outcome depends on timing of concurrent operations

Term	Meaning
Registry (RMI)	RMI naming service on port 1099
Remote (Java)	Marker interface for RMI-callable interfaces
RemoteException	Mandatory exception on RMI methods
ResourceManager	YARN cluster scheduler (port 8032)
RMI	Remote Method Invocation (Java)
Round Robin	Cyclic load-balancing algorithm
RPC	Remote Procedure Call
Selection (GA)	Choosing parents based on fitness
Serialization	Converting objects to bytes
Shuffle and Sort	MapReduce phase grouping values by key
Skeleton	Server-side counterpart to a stub
Stateless	Server keeps no per-client information
Sticky session	Client always routed to the same server
Swarm intelligence	Collective behavior by simple agents
Synchronized (Java)	Method/block guarded by a mutex lock
Synchronous	Caller blocks until result returns
TSP	Travelling Salesman Problem
UnicastRemoteObject	RMI superclass that exports an object
URI (Pyro4)	`PYRO:obj_xxx@host:port`
Writable	Hadoop’s serialization interface
YARN	Yet Another Resource Negotiator

Quick-Reference Cheat Sheet (last-minute revision)

Practical	One-line summary
1 — RPC Factorial	Client calls `fact(n)` on a remote Pyro4 server; server returns n!
2 — RMI String Concat	Java RMI; client invokes `concat`, `reverse`, `upper` on a remote object
3 — Word Count	Hadoop MapReduce: Map emits `(word,1)`, Reduce sums
4 — Fuzzy Sets	Element-wise max (∪), min (∩), 1−x (complement), min(a, 1−b) (difference)
5 — Load Balancing	Round Robin / Random / Least Conn / Weighted distribution of requests
6 — Clonal Selection	Select best, clone, mutate, repeat → converges to target
7 — AIS Damage Classifier	Train antibodies on labelled samples; classify new readings by nearest match
8 — Evolutionary Algorithm	Tournament selection + arithmetic crossover + mutation + elitism
9 — RMI Hotel Booking	Stateful Java RMI service with `synchronized` methods to prevent races
10 — ACO TSP	Ants build tours; pheromone reinforced on short tours, evaporates over time

Concept	Where it appears
RPC paradigm	P1, P2, P9
Marshalling/serialization	P1, P2, P9
Stub/skeleton	P1, P2, P9
At-most-once semantics	P2, P9
Distributed FS / scheduling	P3
Bio-inspired heuristic	P6, P7, P8, P10
Exploration vs exploitation	P6, P8, P10
Concurrency / race conditions	P9
Stateless vs stateful	P5 (stateless), P9 (stateful)
Fault tolerance	P3 (HDFS replication)

End of document. Read each practical's section once carefully, then re-skim the Cross-Cutting Q&A and the cheat sheet right before you go in. Most examiners only have time for 4–6 questions, drawn predominantly from the "Likely viva questions" lists per practical.